

SE: LAB 3: Architecture Selection & Justification Document

System: Public Bus Live Tracking & Crowding Estimator System (Problem Statement #24)

Name: Prajin Ravi Shankar

SRN: PES1UG24CS328

Section: 5-F

ARCHITECTURAL JUSTIFICATION DOCUMENT

1. Architecture Selection

We chose **Microservices Architecture** for the **Public Bus Live Tracking & Crowding Estimator System**.

2. Two Scenario-Related Reasons

1. Decoupling Real-Time Telemetry Ingestion from User Search Queries:

The platform processes high-frequency GPS telemetry every 5 seconds and passenger boarding/alighting counts across 1,000 active buses. Microservices architecture decouples high-throughput data ingestion (GPS & Sensor Ingestion Component) from commuter read traffic (Commuter API Component). This ensures heavy sensor stream processing does not cause latency spikes or UI freezes during commuter route searches.

2. Targeted Independent Scalability for Peak Traffic:

System workloads are highly asymmetrical. Commuter search queries experience heavy traffic spikes during morning and evening rush hours, whereas bus GPS streaming maintains a continuous write load throughout operating hours. Microservices allow independent scaling of the Commuter API Component and Tracking & ETA Service Component without allocating unnecessary compute resources to low-frequency services like Database Component.

3. Security Advantage

Isolation of administrative controls and delay alerting logic (Fleet Monitoring & Alert Component) behind discrete component boundaries and explicit interfaces enforces strict role-based access control. Public mobile app clients interact exclusively with public-facing endpoints on the Commuter API Component and are structurally blocked from directly accessing internal database persistence interfaces (IDataPersistence) or administrative delay configurations.

4. Performance Benefit (Meeting Performance Requirements)

NFR-001 requires handling telemetry streams from up to 1,000 active transit vehicles simultaneously, while NFR-002 mandates that commuter-facing ETA and crowding displays refresh within 3 seconds. Utilizing non-blocking event-driven communication between the ingestion engine, ETA calculation service, and notification handlers eliminates thread blocking. Decoupling these processes guarantees sub-second processing overhead, ensuring live tracking updates easily meet the 3-second UI refresh budget under maximum operational load.